HDOC Day-71 Activity

Dynamic Matrix Operations Using Pointer Notation in C

Name	Dhruv Pandey
Student ID	590037660
Batch	15
Activity	HDOC Day-71

Problem Statement

Write a C program to dynamically allocate two matrices and perform addition, subtraction, and multiplication using pointer notation. Implement each operation as a separate function and release all allocated memory properly.

Objective

To understand dynamic memory allocation for two-dimensional matrices, pointer notation, modular programming using functions, matrix operations, and proper deallocation of heap memory.

Algorithm

1. Read the dimensions of Matrix A and Matrix B.
2. Dynamically allocate memory for both matrices using malloc().
3. Read matrix elements using pointer notation.
4. If dimensions are equal, perform addition and subtraction using separate functions.
5. If columns of A equal rows of B, perform matrix multiplication using a separate function.
6. Display the resulting matrices.
7. Free every dynamically allocated row and finally free each matrix pointer.

C Program

```
#include <stdio.h>
#include <stdlib.h>

int **allocateMatrix(int rows, int cols) {
    int **matrix = (int **)malloc(rows * sizeof(int *));
    for (int i = 0; i < rows; i++)
        *(matrix + i) = (int *)malloc(cols * sizeof(int));
    return matrix;
}

void inputMatrix(int **matrix, int rows, int cols) {
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            scanf("%d", (*(matrix + i) + j));
}

void displayMatrix(int **matrix, int rows, int cols) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++)
            printf("%d\t", *(*(matrix + i) + j));
        printf("\n");
    }
}

int **addMatrices(int **A, int **B, int rows, int cols) {
    int **C = allocateMatrix(rows, cols);
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            *(*(C+i)+j) = *(*(A+i)+j) + *(*(B+i)+j);
```

```
    return C;
}

int **subtractMatrices(int **A, int **B, int rows, int cols) {
    int **C = allocateMatrix(rows, cols);
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            *(*(C+i)+j) = *(*(A+i)+j) - *(*(B+i)+j);
    return C;
}

int **multiplyMatrices(int **A, int **B, int r1, int c1, int c2) {
    int **C = allocateMatrix(r1, c2);
    for (int i = 0; i < r1; i++) {
        for (int j = 0; j < c2; j++) {
            *(*(C+i)+j) = 0;
            for (int k = 0; k < c1; k++)
                *(*(C+i)+j) += *(*(A+i)+k) * *(*(B+k)+j);
        }
    }
    return C;
}

void freeMatrix(int **matrix, int rows) {
    for (int i = 0; i < rows; i++)
        free(*(matrix + i));
    free(matrix);
}

int main() {
    int r1, c1, r2, c2;
    printf("Enter rows and columns of Matrix A: ");
    scanf("%d %d", &r1, &c1);
    printf("Enter rows and columns of Matrix B: ");
    scanf("%d %d", &r2, &c2);

    int **A = allocateMatrix(r1, c1);
    int **B = allocateMatrix(r2, c2);

    printf("Enter elements of Matrix A:\n");
    inputMatrix(A, r1, c1);
    printf("Enter elements of Matrix B:\n");
    inputMatrix(B, r2, c2);

    if (r1 == r2 && c1 == c2) {
        int **sum = addMatrices(A, B, r1, c1);
        int **diff = subtractMatrices(A, B, r1, c1);
        printf("\nAddition:\n");
        displayMatrix(sum, r1, c1);
        printf("\nSubtraction:\n");
        displayMatrix(diff, r1, c1);
        freeMatrix(sum, r1);
        freeMatrix(diff, r1);
    } else {
        printf("\nAddition and subtraction are not possible.\n");
    }

    if (c1 == r2) {
        int **product = multiplyMatrices(A, B, r1, c1, c2);
        printf("\nMultiplication:\n");
        displayMatrix(product, r1, c2);
        freeMatrix(product, r1);
    } else {
        printf("Multiplication is not possible.\n");
    }

    freeMatrix(A, r1);
    freeMatrix(B, r2);
    return 0;
}
```

Sample Execution

```
Enter rows and columns of Matrix A: 2 2
Enter rows and columns of Matrix B: 2 2
Enter elements of Matrix A:
1 2
3 4
Enter elements of Matrix B:
5 6
7 8

Addition:
6       8
10      12

Subtraction:
-4      -4
-4      -4

Multiplication:
19      22
43      50
```

Result

The program successfully dynamically allocates two matrices and performs matrix addition, subtraction, and multiplication using pointer notation. Each operation is implemented in a separate function, and all allocated memory is released using free().

HDOC Day-71 - Program Execution Screenshot

Dhruv Pandey | Student ID: 590037660 | Batch 15

```
Last login: Tue Nov 10 22:37:46 on console
dhruvpandey@DHRUVs-MacBook-Air ~ % nano day71.c
dhruvpandey@DHRUVs-MacBook-Air ~ % clang day71.c -oday71
dhruvpandey@DHRUVs-MacBook-Air ~ % ./day71
Enter rows and columns of Matrix A: 1 2
Enter rows and columns of Matrix B: 3
4

Enter elements of Matrix A:
1
2

Enter elements of Matrix B:
3
54
5
44
44
3
455
6
77
8
8
8

Addition and subtraction are not possible.

Matrix multiplication is not possible.
dhruvpandey@DHRUVs-MacBook-Air ~ % 7
zsh: command not found: 7
dhruvpandey@DHRUVs-MacBook-Air ~ % 7
zsh: command not found: 7
dhruvpandey@DHRUVs-MacBook-Air ~ %
```